On this page

RabbitMQ Segregation for Pulse and Case Manager

This document provides a step-by-step guide for migrating from a single RabbitMQ cluster to two segregated clusters: one dedicated to Pulse and another to Case Manager. The goal is to ensure a transition with no message loss, supporting a zero-downtime deployment.

Migration Overviewâ□□

Migrating RabbitMQ from a single shared cluster to segregated clusters for Pulse and Case Manager requires several steps to ensure zero message loss and minimal downtime. The process includes the following steps:

1. Prepare the new RabbitMQ cluster by creating all the necessary exchanges and queues for Case Manager
2. Enable maintenance mode in Case Manager to ensure no messages are consumed during the process
3. Create a RabbitMQ federation policy to forward messages from the old to the new cluster, preventing data loss and consumption duplication
4. Deploy Case Manager to consume messages from the new cluster
5. Deploy Pulse to publish Case Manager messages to the new cluster
6. Clean up the old Case Manager cluster once migration is complete

The federation step is crucial for a seamless migration, as it ensures that messages published to the old cluster during the transition are not lost and are safely streamed to the new cluster. This is achieved by:

- Setting a one-way policy that replicates messages published to the old RabbitMQ cluster to the new one
- Setting the acknowledge mode to on-confirm, so messages are only consumed a single time

It is expected that during the migration, both clusters will hold messages based on the federation policy above, which should start to be consumed once maintenance mode is disabled.

The diagram below illustrates the expected architecture after the migration:

Migration Stepsâ□□

In this section you can find a description of the migration steps.

Prepare the new RabbitMQ clusterâ□□

The first step is to prepare the new cluster, ensuring it has all the necessary queues/exchanges that Case Manager requires. This can be done in several ways, for instance:

- Creating a temporary Case Manager cluster, configured to use the new cluster, and decommissioning it afterwards
- Creating all the resources manually using the RabbitMQ Management UI
- Using `rabbitmqctl` to list and create all the necessary resources

warning

To avoid message loss it is crucial that all queues/exchanges are created in the new RabbitMQ cluster before enabling the federation policy.

If the first option is used make sure to remove the temporary cluster before creating the federation policy to ensure no messages are consumed.

Enable maintenance mode in Case Managerâ

To ensure that no messages are processed during the migration, maintenance mode should be enabled in Case Manager. By enabling maintenance mode it is guaranteed that messages are only consumed by the new cluster once the migration is concluded.

Set Up Federationâ

Setting up a federation policy to replicate all Case Manager messages to the new cluster guarantees that no messages are lost in the process and that each is only consumed once. The process is described as follows:

- Enable the federation plugin on both clusters:

```
rabbitmq-plugins enable rabbitmq_federation
rabbitmq-plugins enable rabbitmq_federation_management
```

Copy

- On the new cluster, configure a federation upstream policy to connect to the old cluster. The 'ack-mode' is set to 'on-confirm' on the configuration of the federation, ensuring messages are only removed from the old cluster after being safely received and consumed by the new cluster. This establishes a link so the new cluster can pull messages from the old cluster during migration:
  - `<new-vhost>` : the vhost in the new Case Manager RabbitMQ cluster. It can be the same vhost name as in the old cluster
  - `<user>/<pass>` : RabbitMQ credentials with access to the old cluster
  - `<old-cluster>` : hostname or IP of the old cluster
  - `<old-vhost>` : the vhost used by Case Manager on the old cluster

```
rabbitmqctl set_parameter -p <new-vhost> federation-upstream cm-federation '{"uri":
["amqp://<user>:<pass>@<old-cluster>/<old-vhost>"], "ack-mode":"on-confirm"}'
```

Copy

- Apply a federation policy on the new cluster to specify which exchanges should receive messages from the old cluster via federation. Federate only the exchanges used by Case Manager (`CM_UPDATES_CASES` , `CM_UPDATES_EVENTS` , and `EVENT_QUEUE_.*`):

```
rabbitmqctl set_policy -p <new-vhost> --apply-to exchanges cm-federation-policy
'^(CM_UPDATES_CASES|CM_UPDATES_EVENTS|EVENT_QUEUE_.*)$" '{"federation-upstream":"cm-
federation"}'
```

Copy

warning

Ensure that `<new-vhost>` matches the vhost configured via Case Manager Ansible variables for the new cluster. For Pulse (Alert Storage Service), use the vhost set by the corresponding Pulse Ansible variable.

The diagram below illustrates the federation setup between the old and new RabbitMQ clusters. In particular, this shows how the architecture will look when both Pulse and Case Manager are still connected to the old RabbitMQ clusters, but after the new RabbitMQ cluster has been created and the federation link has been set up

Deploy Pulse and Case Managerâ

Following the federation setup, both Pulse and Case Manager should be deployed. The deployment is required so Pulse publishes messages to the new RabbitMQ cluster and Case Manager consumes messages from the new cluster.

To achieve this, the Ansible variables file should be updated for both components:

- Update configuration variables:
  - `rabbitmq_casemanager_brokers` : set to the new RabbitMQ cluster brokers
  - `rabbitmq_casemanager_vhost` : set to the new RabbitMQ cluster vhost

- Deploy Case Manager to the new RabbitMQ cluster. It will start consuming messages from the new cluster and pulling any remaining messages from the old cluster via federation
    - Before disabling maintenance mode on the new Case Manager deployment, stop/decommission the old Case Manager cluster. This ensures only the new cluster is active and prevents message consumption conflicts
- Deploy Pulse which will start publishing messages to the new cluster
- With federation configured using `ack-mode: on-confirm`, the federation link acts as the consumer for the old cluster, safely ensuring messages are only removed from the old cluster after being safely received and consumed by the new cluster

Please refer to Case Manager Configuration Reference and Pulse Configuration Reference for more information.

Cutover and cleanâ□□

warning

Make sure all Case Manager queues in the old RabbitMQ cluster are drained before proceeding with the cleanup.

Once both components are deployed and using the correct RabbitMQ cluster, any leftover configurations/resources can be safely removed:

- Remove federation policies and upstreams from the new cluster:

```
rabbitmqctl clear_policy -p <new-vhost> cm-federation-policy
rabbitmqctl clear_parameter -p <new-vhost> federation-upstream cm-federation
```

Copy

- Clean the old cluster Case Manager exchanges and queues:

```
rabbitmqctl delete_exchange -p <old-vhost> CM_UPDATES_CASES
rabbitmqctl delete_exchange -p <old-vhost> CM_UPDATES_EVENTS
rabbitmqctl delete_queue -p <old-vhost> EVENT_QUEUE_.*
```

Copy